Solution: House Robber II

Problem Statement

You are given an array representing the amount of money each house has. This array models a circle of houses, meaning that the first and last houses are adjacent. You are tasked with figuring out the maximum amount of money you can rob without alerting the neighbors.

The rule is: if you rob one house, you cannot rob its adjacent houses.

Example Generation

Example 1:

- **Input**: [4, 2, 3, 1]
- **Expected Output**: 7
- **Justification**: Rob the 1st and 3rd house, which gives 4 + 3 = 7.

Example 2:

- **Input**: [5, 1, 2, 5]
- **Expected Output**: 7
- **Justification**: Rob the 1st and 3rd house, which gives 5 + 2 = 7.

Example 3:

- **Input**: [1, 2, 3, 4, 5]
- **Expected Output**: 8
- **Justification**: Rob the 3rd and 5th house, which gives 3 + 5 = 8.

Solution

The core idea of our algorithm is to split the circular house problem into two simpler linear problems and then solve each linear problem using a dynamic programming approach. Since the first and last houses in the circle are adjacent, we can't rob them both. Thus, we consider two scenarios: robbing houses excluding the first one and robbing houses excluding the last one. For each scenario, we use a helper function that returns the maximum amount we can rob. This helper function utilizes dynamic programming to keep track of the cumulative robbery amounts, ensuring that no two consecutive houses are robbed. Finally, our solution is the maximum of the two scenarios.

1. **Edge Cases Handling:** Begin by checking for edge cases:
    - If the input array is `null` or has a length of 0, return 0 since there are no houses to rob.
    - If there's only one house, return its value.
    - If there are two houses, return the maximum value of the two houses.
2. **Two Scenarios Handling:** Due to the circular structure, handle two scenarios:
    - Exclude the first house and compute for the rest.
    - Exclude the last house and compute for the others.

    Use a helper function to compute the maximum for each scenario.
3. **Simple Robber Helper Function:** This function calculates the maximum robbing amount for a linear set of houses using dynamic programming:
    - Maintain two variables, `prevMax` and `currMax`, to keep track of the max amount robbed up to the previous and current house, respectively.
    - Iterate through each house in the given range (based on the scenario). At each house, decide whether to rob it (in which case you can't rob the previous house) or skip it.
    - For each house, update `currMax` based on whether robbing the current house results in a larger amount than skipping it.
4. **Return the Maximum:** The main function concludes by returning the maximum value from the two scenarios.

Algorithm Walkthrough

Using the input [4, 2, 3, 1]:

1. Check for edge cases. Since the array has more than two houses, move to the next step.
2. Calculate the maximum amount that can be robbed for the two scenarios:

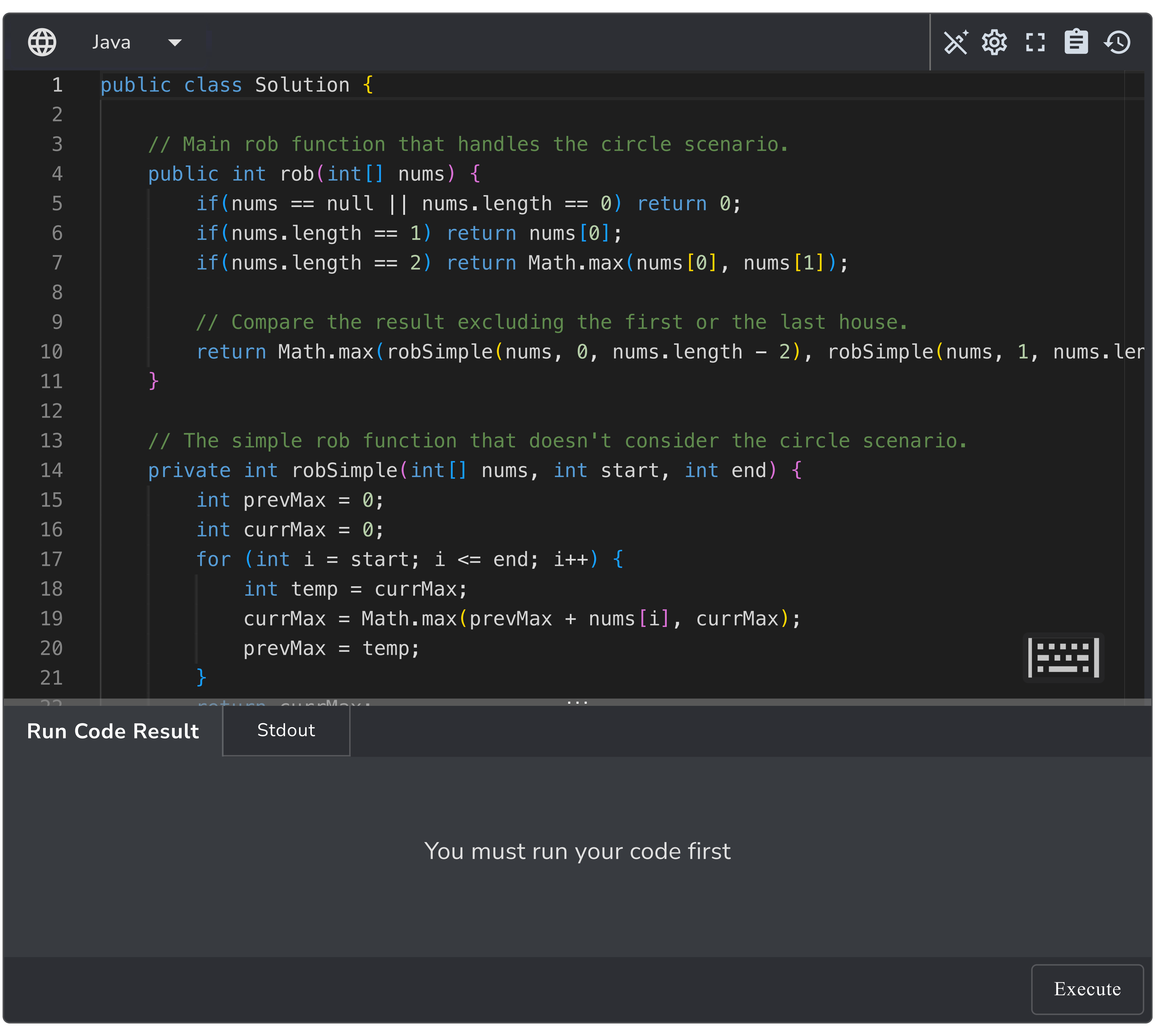
    a. For the scenario excluding the last house (consider houses from index 0 to 2):

    - Start with `prevMax = 0` and `currMax = 0`.
    - For the first house (value 4), `currMax` becomes 4.
    - For the second house (value 2), `currMax` remains 4 (because 4 > 2 + 0).
    - For the third house (value 3), `currMax` becomes 7 (because 4 + 3 > 4). So, for this scenario, the maximum is 7.

    b. For the scenario excluding the first house (consider houses from index 1 to 3):

    - Start with `prevMax = 0` and `currMax = 0`.
    - For the second house (value 2), `currMax` becomes 2.
    - For the third house (value 3), `currMax` becomes 3 (because 3 > 2).
    - For the fourth house (value 1), `currMax` remains 3 (because 3 > 1 + 2). So, for this scenario, the maximum is 3.
3. The main function then returns the maximum of the two scenarios, which is 7 in this case.

Code



```java
public class Solution {

    // Main rob function that handles the circle scenario.
    public int rob(int[] nums) {
        if(nums == null || nums.length == 0) return 0;
        if(nums.length == 1) return nums[0];
        if(nums.length == 2) return Math.max(nums[0], nums[1]);

        // Compare the result excluding the first or the last house.
        return Math.max(robSimple(nums, 0, nums.length - 2), robSimple(nums, 1, nums.len
    }

    // The simple rob function that doesn't consider the circle scenario.
    private int robSimple(int[] nums, int start, int end) {
        int prevMax = 0;
        int currMax = 0;
        for (int i = start; i <= end; i++) {
            int temp = currMax;
            currMax = Math.max(prevMax + nums[i], currMax);
            prevMax = temp;
        }
```

You must run your code first

Complexity Analysis

Time Complexity: We are solving the house robber problem twice (once excluding the first house and once excluding the last house). Each run of the house robber problem has a time complexity of (O(n)), where (n) is the number of houses. Thus, our overall time complexity is (O(n)).

Space Complexity: We use a constant amount of space to store our previous and current max values. Hence, the space complexity is (O(1)).

← Previous
House Robber II (Medium)

Next →
Decode Ways (Medium)

☐ Mark as Completed